CogLean

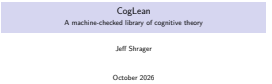
A machine-checked library of cognitive theory

Jeff Shrager

October 2026

2026-11-02

CogLean



This talk is about CogLean, a way of stating cognitive theories so that what follows from them can be checked, compared and accumulated. The vehicle is Lean, a programming language that is also a proof checker, but you won't need to read any Lean today. I'll describe the problem, what the library contains, how it is organized around laws rather than individual models, two examples, a related approach, and where it falls short of mathematics. The library is open source, on GitHub.

The problem

Cognitive theories are stated in prose, in simulation code, or as equations fitted to data.

- What follows from a theory's *core assumptions*, as distinct from its parameter settings or implementation details, is rarely checked.
- A simulation shows that a model *can* produce a result, with these parameters and these stimuli.
- It doesn't show which assumptions produced it, whether it survives other parameters, or whether a rival theory would produce it just as well.

Most of our theories live in prose, in simulation code, or as equations fitted to data. When we evaluate them, we usually run a model with particular parameters on particular stimuli and compare the output with data. That tells us the theory can produce a result. It doesn't tell us which of the theory's assumptions produced it, whether it would survive a different choice of parameters, or whether a rival theory would produce it just as well. Those are exactly the questions we care about when we compare theories, and they are rarely answered.

The problem

Cognitive theories are stated in prose, in simulation code, or as equations fitted to data.

- What follows from a theory's *core assumptions*, as distinct from its parameter settings or implementation details, is rarely checked.
- A simulation shows that a model *can* produce a result, with these parameters and these stimuli.
- It doesn't show which assumptions produced it, whether it survives other parameters, or whether a rival theory would produce it just as well.

Three kinds of evidence a theory can give

One run	**Every run**	**Every model satisfying the postulates**
one model, one parameter setting	one model, all its executions	all models the theory allows
the theory *can* do it	that model *always* does it	the postulates *entail* it
simulation	model checking, sweeps	proof

CogLean is built around the third kind, and keeps the other two in the same language.

Three kinds of evidence a theory can give

One run	Every run	Every model satisfying the postulates
one model, one parameter setting	one model, all its executions	all models the theory allows
the theory *can* do it	that model *always* does it	the postulates *entail* it
simulation	model checking, sweeps	proof

CogLean is built around the third kind, and keeps the other two in the same language.

It helps to separate three kinds of evidence. One run: one model, one parameter setting; the theory can do it. Every run: one model in all its executions, by exhaustive search or parameter sweeps; that model always does it. And every model the theory allows: the postulates entail it. That third kind is a proof, and it's what CogLean is built around. A proof tells you which assumptions matter, because it uses some postulates and not others, and it distinguishes what a theory can do from what it must do. Simulation stays essential for exploring and fitting, and experiments with people still decide whether the assumptions are right.

Why Lean

- **One definition, run and proved.** The same definition of a memory can be *run* on an experiment's stimuli and *proved about* for all stimuli. The simulation and the theory can't drift apart.
- **Checked.** Every proof step is checked by a small trusted kernel.
- **General.** A theorem can quantify over all nets, all parameter values, all stimulus lists.
- **Cumulative.** Like Mathlib for mathematics: definitions made once, theorems imported rather than re-derived.

The guarantee is relative. Lean checks that a theorem follows from the definitions, not that the definitions capture the theory, still less that the theory is true of people.

Why Lean

- **One definition, run and proved.** The same definition of a memory can be *run* on an experiment's stimuli and *proved about* for all stimuli. The simulation and the theory can't drift apart.
- **Checked.** Every proof step is checked by a small trusted kernel.
- **General.** A theorem can quantify over all nets, all parameter values, all stimulus lists.
- **Cumulative.** Like Mathlib for mathematics: definitions made once, theorems imported rather than re-derived.

The guarantee is relative. Lean checks that a theorem follows from the definitions, not that the definitions capture the theory, still less that the theory is true of people.

We use Lean, which is both a programming language and a proof checker. That matters because the same definition, say of a discrimination net, can be run on an experiment's stimuli and proved about for all stimuli, so the simulation and the theory can't drift apart. Every proof step is checked by a small trusted kernel. Theorems can quantify over all parameter values. And the library is cumulative: definitions are made once and reused. One caution, which I'll come back to: Lean checks that theorems follow from the definitions, not that the definitions capture the theory.

What is in the library

- **CogLib**: shared constructs (feature codes, exact probability distributions, discrimination nets, sampled retrieval, attention) and theory-neutral phenomena.
- **EPAM** (Feigenbaum): discrimination-net learning. Stimulus generalization, oscillation, retroactive inhibition, learning to criterion, the serial position effect: all proved. A published serial-anticipation run is reproduced (33–34 of 34 responses agree).
- **Strategy choice** (Siegler & Shrager): distributions of associations. Retrieval is used with probability $1 - (1 - R)^n$; and a finding: a more peaked distribution can *raise* overt strategy use.
- **Multi-digit addition**: the two combined, with 18 predictions neither makes alone. Built as an **application**: library mechanisms wired together (recognize, retrieve, else solve the columns with single-digit trials), with no new mechanism and no answer given.

What is in the library

- **CogLib**: shared constructs (feature codes, exact probability distributions, discrimination nets, sampled retrieval, attention) and theory-neutral phenomena.
- **EPAM** (Feigenbaum): discrimination-net learning. Stimulus generalization, oscillation, retroactive inhibition, learning to criterion, the serial position effect: all proved. A published serial-anticipation run is reproduced (33-34 of 34 responses agree).
- **Strategy choice** (Siegler & Shrager): distributions of associations. Retrieval is used with probability $1 - (1 - R)^k$; and a finding: a more peaked distribution can raise overt strategy use.
- **Multi-digit addition**: the two combined, with 18 predictions neither makes alone. Built as an **application**: library mechanisms wired together (recognize, retrieve, else solve the columns with single-digit trials), with no new mechanism and no answer given.

Here's what is in the library now. CogLib holds the shared constructs: feature codes, exact probability distributions, discrimination nets, sampled retrieval and attention, along with phenomena stated without reference to any theory. On top of it sit three theories. EPAM, Feigenbaum's discrimination-net model, where stimulus generalization, oscillation, retroactive inhibition, learning to criterion and the serial position effect are all proved, and a published simulation run is reproduced. The distributions-of-associations model of strategy choice, where proof turned up something surprising: a more peaked distribution can actually raise overt strategy use. And a model of multi-digit addition that combines the two. That last one is really an application: it's built by wiring library mechanisms together, recognition, then retrieval, else a problem solver that works the columns with single-digit trials, with no new mechanism and nothing handed the answer.

The generality problem

There are a great many specific models: EPAM, ACT-R, SAM, REM, MINERVA 2, TODAM, connectionist networks...

- Formalizing them one at a time doesn't scale.
- A pile of formalized models doesn't say what any two of them *disagree* about.
- And none of them is "true": unlike mathematics, there's no settled foundation to build on.

The generality problem

There are a great many specific models: EPAM, ACT-R, SAM, REM, MINERVA 2, TODAM, connectionist networks. . .

- Formalizing them one at a time doesn't scale.
- A pile of formalized models doesn't say what any two of them *disagree* about.
- And none of them is "true": unlike mathematics, there's no settled foundation to build on.

Everything so far is about one model. But there are a great many models, such as EPAM, ACT-R, SAM, REM, MINERVA and connectionist networks, and formalizing them one at a time doesn't scale. Worse, a pile of formalized models doesn't tell you what any two of them disagree about. And unlike mathematics, none of them is simply true. So the question is how a general library should be organized.

Laws and families

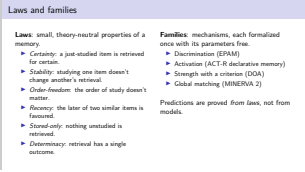
Laws: small, theory-neutral properties of a memory.

- *Certainty*: a just-studied item is retrieved for certain.
- *Stability*: studying one item doesn't change another's retrieval.
- *Order-freedom*: the order of study doesn't matter.
- *Recency*: the later of two similar items is favoured.
- *Stored-only*: nothing unstudied is retrieved.
- *Determinacy*: retrieval has a single outcome.

Families: mechanisms, each formalized once with its parameters free.

- Discrimination (EPAM)
- Activation (ACT-R declarative memory)
- Strength with a criterion (DOA)
- Global matching (MINERVA 2)

Predictions are proved *from laws*, not from models.



The answer is to organize the library the way Mathlib organizes algebra. Mathlib doesn't formalize every group; it states laws, like those of a monoid or a ring, proves theorems once per law, and lets particular structures inherit them. For memory, the laws are small properties a memory may or may not have: certainty, stability, order-freedom, recency, stored-only retrieval and determinacy. The families are mechanisms, each formalized once with its parameters free: discrimination, activation, strength with a criterion, and global matching. Predictions are then proved from laws, not from particular models.

The theory matrix

	Certainty	Stability	Order-free	Recency	Stored-only	Determinate
Discrimination	**has**	**has**	lacks	lacks	**has**	**has**
Activation	**never**	**never**	**never**	**has**	**has**	**never**
Strength	lacks	lacks	**has**	**never**	**has**	lacks
Global matching	lacks	lacks	**has**	**never**	lacks	**has**

Every cell is a theorem: a general proof that the law holds (**has**) or fails (**never**) at every parameter setting, or a counterexample at one setting (lacks). Every law separates some families, and every family has its own row.

The theory matrix

	Certainty	Stability	Order-free	Recency	Stored-only	Determinate
Discrimination	**has**	**has**	lacks	lacks	**has**	**has**
Activation	**never**	**never**	**never**	**has**	**has**	**never**
Strength	lacks	lacks	**has**	**never**	**has**	lacks
Global matching	lacks	lacks	**has**	**never**	lacks	**has**

Every cell is a theorem: a general proof that the law holds (**has**) or fails (**never**) at every parameter setting, or a counterexample at one setting (lacks). Every law separates some families, and every family has its own row.

Put the two together and you get a matrix. Every cell is a theorem. Has means a general proof that the family has the law at every parameter setting meeting the stated conditions; never, in bold red, means a general proof that it fails at every such setting; and lacks, in plain red, is a counterexample at one setting. The difference matters, because only the every-setting cells can be refuted by evidence. Every law separates some families, and every family has its own row. Global matching was added as a test of the laws, and on the first four it looked exactly like strength memory, which forced the stored-only law. Some of these laws are stated too narrowly, all-or-none where they should be graded; fixing that is the next piece of work.

What is true here?

In mathematics, a proved theorem is true. Here there are only competing theories. What can a library build on?

Claim	Example	Status
A law entails a prediction	stability $\Rightarrow$ practiced items stay practiced	proved, permanent
A family has a law	discrimination nets have stability	proved, permanent
People have a law	people lack stability in recall	empirical, revisable

Which theory is right follows from the second and third rows together.

What is true here?

In mathematics, a proved theorem is true. Here there are only competing theories. What can a library build on?

Claim	Example	Status
A law entails a prediction	stability ⇒ practiced items stay practiced	proved, permanent
A family has a law	discrimination nets have stability	proved, permanent
People have a law	people lack stability in recall	empirical, revisable

Which theory is right follows from the second and third rows together.

In mathematics, a proved theorem is true. Here we have competing theories, so what is true? There are three kinds of claim. That a law entails a prediction is proved, and permanent. That a family has a law is proved, and permanent. Whether people have a law is empirical, and revisable. Which theory is right follows from the second and third together, and the first two never expire, even if every model in the library is refuted.

A refutation spreads

- If people lack a law, every family that has it at every setting is refuted at once, and every prediction proved from it loses its support, in every theory that uses it. A counterexample at one setting refutes nothing.
- The matrix carries a judged evidence row, kept apart from the theorems.
- Each judgment states its **bridge**: the assumption connecting what experiments measure (reports, decisions) to what a law is about (what memory retrieves).

Example. People endorse a prototype they never saw. That does not show they *retrieved* it: a memory that returns a similar studied item also calls the prototype "old". The bridge is the missing piece.

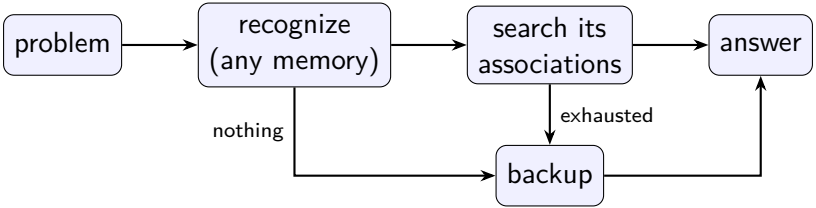
A refutation spreads

- If people lack a law, every family that has it at every setting is refuted at once, and every prediction proved from it loses its support, in every theory that uses it. A counterexample at one setting refutes nothing.
- The matrix carries a judged evidence row, kept apart from the theorems.
- Each judgment states its **bridge**: the assumption connecting what experiments measure (reports, decisions) to what a law is about (what memory retrieves).

Example. People endorse a prototype they never saw. That does not show they retrieved it: a memory that returns a similar studied item also calls the prototype "old". The bridge is the missing piece.

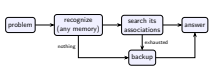
This structure has a useful property: a refutation spreads. If the evidence shows that people lack a law, every family that has the law at every setting is refuted at once, and every prediction proved from it loses its support, in every theory that uses it. A counterexample at one setting refutes nothing, since another setting may do what people do. So the matrix carries an evidence row for people, kept apart from the theorems. Each entry states its bridge, the assumption that connects what an experiment measures, which is a report, to what a law is about, which is retrieval. Endorsing a prototype you never saw, for instance, doesn't show you retrieved it. Often the next step in comparing theories is not more data but an explicit response model.

Example 1: confident errors, inherited



- **Theorem**: a stranger taken for a familiar item with sharp associations is answered wrongly every time an answer is retrieved, with no more backup use than the familiar item.
- Proved once, for the composition. Inherited by multi-digit addition ($150 + 100$ taken for $100 + 100$) and by face naming (a stranger misnamed).
- From laws: any memory with determinacy, certainty and stability gives it. In the matrix, only discrimination.

Example 1: confident errors, inherited



- **Theorem**: a stranger taken for a familiar item with sharp associations is answered wrongly every time an answer is retrieved, with no more backup use than the familiar item.
- Proved once, for the composition. Inherited by multi-digit addition ($150 + 100$ taken for $100 + 100$) and by face naming (a stranger misnamed).
- From laws: any memory with determinacy, certainty and stability gives it. In the matrix, only discrimination.

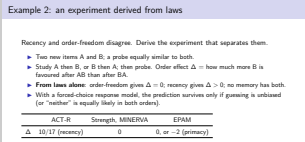
Here is the first example. This composition, recognize with any memory, then search the recognized item's associations, else fall back on a backup, comes from our model of multi-digit addition. For it we proved a theorem about confident errors: a stranger taken for a familiar item with sharp associations is answered wrongly every time an answer is retrieved, with no more backup use than the familiar item itself. It's proved once and inherited by two tasks: 150 plus 100 taken for 100 plus 100, and a stranger misnamed as a familiar person. And from laws: any memory with determinacy, certainty and stability gives it, which in the matrix means only discrimination.

Example 2: an experiment derived from laws

Recency and order-freedom disagree. Derive the experiment that separates them.

- Two new items A and B; a probe equally similar to both.
- Study A then B, or B then A; then probe. Order effect Δ = how much more B is favoured after AB than after BA.
- **From laws alone**: order-freedom gives $\Delta = 0$; recency gives $\Delta > 0$; no memory has both.
- With a forced-choice response model, the prediction survives only if guessing is unbiased (or "neither" is equally likely in both orders).

	ACT-R	Strength, MINERVA	EPAM
Δ	10/17 (recency)	0	0, or -2 (primacy)



The second example derives an experiment from laws. Recency and order-freedom disagree, so take two new items and a probe equally similar to both, study them in either order, and measure how much more the later item is favoured. From the laws alone, order-freedom predicts no effect, recency predicts a positive one, and no memory can have both. With a forced-choice response model, recency's prediction survives only if guessing is unbiased. And across the four families: ACT-R shows recency, strength and MINERVA show nothing, and EPAM, which has neither law, can show the most extreme primacy.

A related approach: SFT-GMA

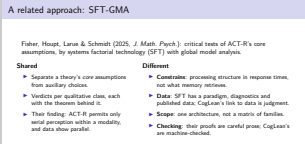
Fisher, Houpt, Larue & Schmidt (2025, *J. Math. Psych.*): critical tests of ACT-R's core assumptions, by systems factorial technology (SFT) with global model analysis.

Shared

- Separate a theory's *core* assumptions from auxiliary choices.
- Verdicts per qualitative class, each with the theorem behind it.
- Their finding: ACT-R permits only serial perception within a modality, and data show parallel.

Different

- **Constrains**: processing structure in response times, not what memory retrieves.
- **Data**: SFT has a paradigm, diagnostics and published data; CogLean's link to data is judgment.
- **Scope**: one architecture, not a matrix of families.
- **Checking**: their proofs are careful prose; CogLean's are machine-checked.



We're not alone in wanting this. Fisher, Houpt, Larue and Schmidt, in the Journal of Mathematical Psychology this year, derived critical tests of ACT-R from eight core assumptions they propose, such as one production at a time, using systems factorial technology with global model analysis. They prove that ACT-R can only process two stimuli in the same modality serially, and published experiments show parallel processing, so some core assumption has to go. Like us, they separate core from auxiliary assumptions and give a verdict per class with its theorem; their tables look like one column of our matrix. The differences: they constrain processing structure seen in response times, we constrain what memory retrieves; their space comes with a paradigm, diagnostics and data, and ours doesn't yet; they analyze one architecture, we compare families; and their proofs are careful prose, ours are machine-checked. They also suggest that base-level activation should be monotone in uses and recency, just the graded law we need next. In the long run, I think executable, machine-checked, and a library that accumulates will win out, but only if it connects to data as well as SFT does. So I expect convergence.

Where this is weaker than mathematics

- **Definitions are contestable.** A family is one formalization of a theory. We check that link with blind simulations and community implementations.
- **Laws are idealizations** and presuppose a representation (binary codes, feature-count similarity).
- **Families are flexible.** The informative facts are parameter regions, not single counterexamples.
- **Evidence is never final.** The empirical row will change; the proved rows won't, and the matrix says at once what follows.

Where this is weaker than mathematics

- **Definitions are contestable.** A family is one formalization of a theory. We check that link with blind simulations and community implementations.
- **Laws are idealizations** and presuppose a representation (binary codes, feature-count similarity).
- **Families are flexible.** The informative facts are parameter regions, not single counterexamples.
- **Evidence is never final.** The empirical row will change; the proved rows won't, and the matrix says at once what follows.

Let me be clear about where this is weaker than mathematics. The definitions are contestable, which is why we do blind checks. The laws are idealizations, and they assume a representation. The families are flexible, so the most informative facts are about regions of their parameters, not single counterexamples. And the evidence is never final. But when it changes, the matrix tells you immediately what has been refuted.

Next

- **Learning across trials**: feedback updates memory, so memory states become random; multi-digit learning sequences are the target.
- **Graded laws**, and the parameter regions of the "lacks" cells.
- **More families as tests**: REM, a connectionist memory, CHREST, ACT-R with blending.
- **Observational semantics**: decision and report components, so laws about retrieval predict what participants say.
- **Beyond memory**: strategy choice, then production systems.

Next steps: learning across trials, where memory states themselves become random; graded laws, which will also tell us where in its parameters each family has each law; more families as tests, such as REM, a connectionist memory, and ACT-R with blending; decision and report components, so that laws about retrieval predict responses; and beyond memory, to strategy choice and production systems.

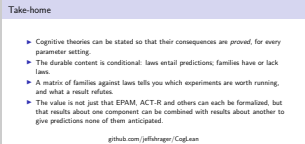
Next

- **Learning across trials**: feedback updates memory, so memory states become random; multi-digit learning sequences are the target.
- **Graded laws**, and the parameter regions of the "lacks" cells.
- **More families as tests**: REM, a connectionist memory, CHREST, ACT-R with blending.
- **Observational semantics**: decision and report components, so laws about retrieval predict what participants say.
- **Beyond memory**: strategy choice, then production systems.

Take-home

- Cognitive theories can be stated so that their consequences are *proved*, for every parameter setting.
- The durable content is conditional: laws entail predictions; families have or lack laws.
- A matrix of families against laws tells you which experiments are worth running, and what a result refutes.
- The value is not just that EPAM, ACT-R and others can each be formalized, but that results about one component can be combined with results about another to give predictions none of them anticipated.

github.com/jeffshrager/CogLean



So, the take-home. Cognitive theories can be stated so that their consequences are proved, for every parameter setting. The durable content is conditional: laws entail predictions, and families have or lack laws. A matrix of families against laws tells you which experiments are worth running and what a result refutes. And the real value isn't that each theory can be formalized; it's that results about one component can be combined with results about another into predictions that no single model anticipated. Thank you.